PROBLEMI COMPLESSI

Esistono 2 tipi di problemi:

- Semplici
- Complessi
  Un algoritmo non ha immediata soluzione, essa deve avvenire tramite fasi di analisi e scelte specifiche.

SOLUZIONE AI PROBLEMI COMPLESSI:

- Scomposizione in sottoproblemi
- Individuazione del procedimento che porta alla soluzione più efficace

APPROCCIO

Creare relazioni tra i sottoproblemi, i sottoproblemi a loro volta possono essere divisi in altri sottoproblemi.
Ci basiamo sul principio del **Divide et Impera**:

- Ridurre la complessità del problema fino ad arrivare a problemi primitivi risolvibili.
  I sottoproblemi si risolvono con tecnica **TOP DOWN DESIGN** o *STEP WISE REFINEMENT*, uno dei principali principali della programmazione, esso ci consente di passare da un problema di complessità *n* ad una gerarchia di problemi con complessità sempre più inferiore a *n*
  La gerarchia dei problemi hanno una relazione **gerarchica** di tipo **padre figlio**.

"Nel processo di raffinamento inizialmente l'attenzione è rivolta a ***cosa*** poi, man mano, raffinando si passa a ***come*** ovvero un algoritmo che risolva il problema."

REQUISITI

Ogni passo deve garantire che:

- La soluzione dei sottoproblemi conduca alla risoluzione del problema principale
- La successione dei passi da seguire abbia senso
- I sottoproblemi siano più vicino agli strumenti disponibili

QUANDO FERMARSI?

- Quando tutti i problemi sono primitivi
- È fissato l'ordine dei sottoproblemi
- È previsto il modo in cui un sottoproblema utilizzerà i risultati per cordinarli ad un altro sottoproblema.

I problemi ottenuti dalla scomposizione sono:

- **INDIPENDENTI** --> Si può progettare un algoritmo per ciascuno
- **COOPERANTI** --> Ciascun sottoproblema può utilizzare le soluzione degli altri
  Quanto più è complesso un problema, tanti saranno i livelli necessari in profondità (generalmente 2 o 3)

ALTERNATIVA AL TOP DOWN

Si chiama sviluppo **BOTTOM-UP**, esso è il metodo opposto al top down perchè lavora in maniera dal basso verso l'alto.
Non è ingegnoso come sistema rispetto al TOP-DOWN, ma è utile per analizzare programmi già scritti e modificarli e/o correggerli (ex. Quando si hanno parti di codice da riutilizzare); entrambe ci consento di creare scomposizioni ben strutturate.

STRUMENTI
Sia per top-down che per bottom-up si usano dei costrutti, offerti da linguaggi di programmazione strutturata che supportano le scomposizioni:

- **Sequenza** --> Suddivide il problema in parti distinte.
- **Selezione** --> Trattamento differente in casi differenti. La soluzione si basa su scelte multiple, cioè strutture di selezione interne ad altre strutture di selezione; ciò si basa sul principio dell'*annidamento* di strutture.
- **Iterazione-->** Successione di problemi dello stesso tipo, Si ripete un numero finito di volte un istruzione per giungere alla fine. La scomposizione iterattiva porta alla soluzione di due problematiche:
  - Quando i sottoproblemi sono uguali differiscono solamente per i dati su cui agiscono
  - I dati su cui agiscono i sottoproblemi sono in relazione d'ordine per questo un sottoproblema differisce dal precedente solo perché usa dato successivo.
- **Ricorsiva** --> Un algoritmo che è definita nei termini di se stessa.

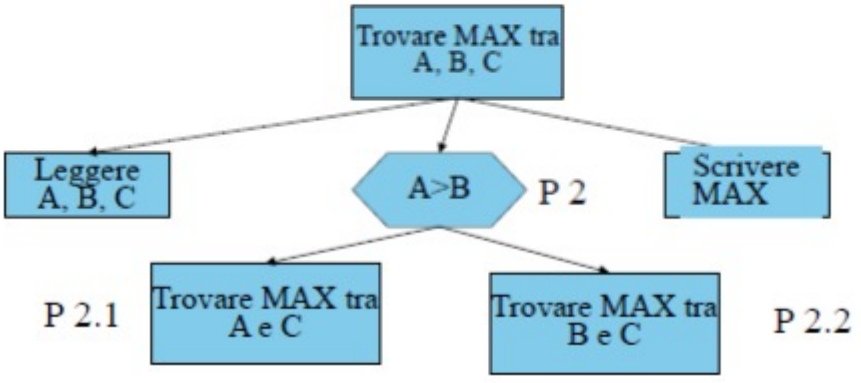
ESEMPIO DI SEQUENZIALE:

- ✓ Problema: Preparare una tazza di tè
- ✓ Soluzione: l'algoritmo per la tazza di tè
  - Bollire l'acqua
  - Mettere il tè nella tazza
  - Versare l'acqua nella tazza
  - Lasciare in infusione per 3 minuti
- ✓ Non sono problemi primitivi
  - Ciascuno va ulteriormente scomposto

1.
   1.1 Riempire d'acqua il bollitore
   1.2 Accendere il gas
   1.3 Aspettare che l'acqua bolla
   1.4 Spegnere il gas
2.
   2.1 Aprire la scatola del tè
   2.2 Prendere un sacchetto-filtro
   2.3 Chiudere la scatola del tè
   2.4 Mettere il sacchetto nella tazza
3.
   3.1 Versare l'acqua bollente nella tazza finché non è piena
4.
   4.1 Aspettare 3 minuti
   4.2 Estrarre il sacchetto-filtro

ESEMPIO DI SELEZIONE:

- ✓ Problema: Trovare il più grande dei numeri *a*, *b*, *c* diversi tra loro
  - Azione primitiva (nota):
    - Verificare se un numero è maggiore di un altro



ESEMPIO DI ITERAZIONE:

- ✓Trovare il più grande fra n > 3 numeri tutti diversi fra loro
  - Supponiamo che i dati siano in relazione d'ordine
    - Si può parlare di 1° dato, 2° dato, ... n° dato

Esempio

- ✓Trova il più grande tra i primi 2 numeri
- ✓Trova il più grande fra il risultato precedente e il 3° numero
- ✓...
- ✓Trova il più grande fra il risultato precedente e l'ultimo numero (*n*° dato)

- ✓Più concisamente:
  - Trova il più grande fra i primi 2 numeri
  - **Mentre** ci sono numeri da esaminare **esegui**
    - Esamina il primo numero non ancora considerato
    - Trova il più grande tra questo e il più grande precedentemente trovato